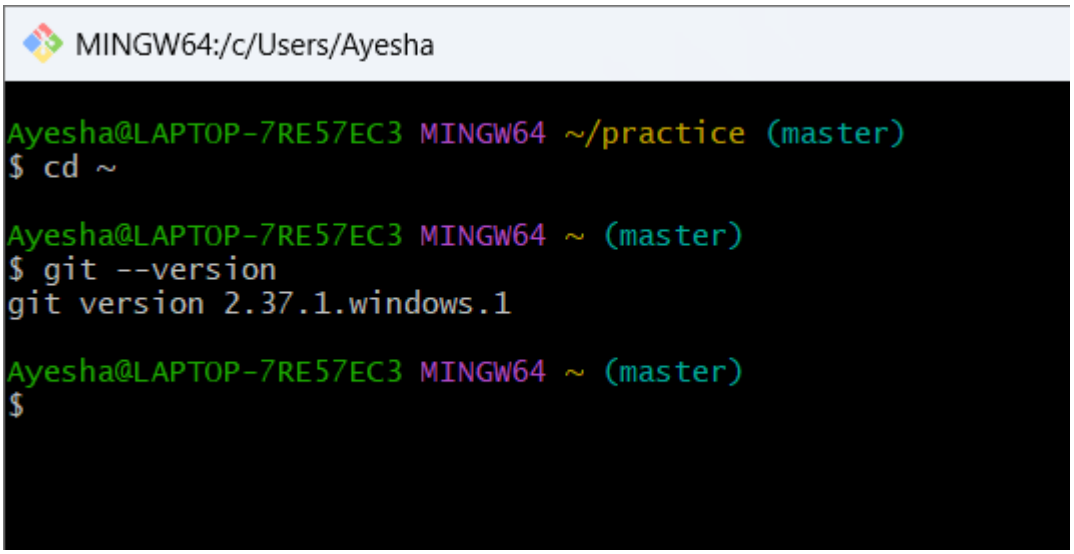
1. Install Git



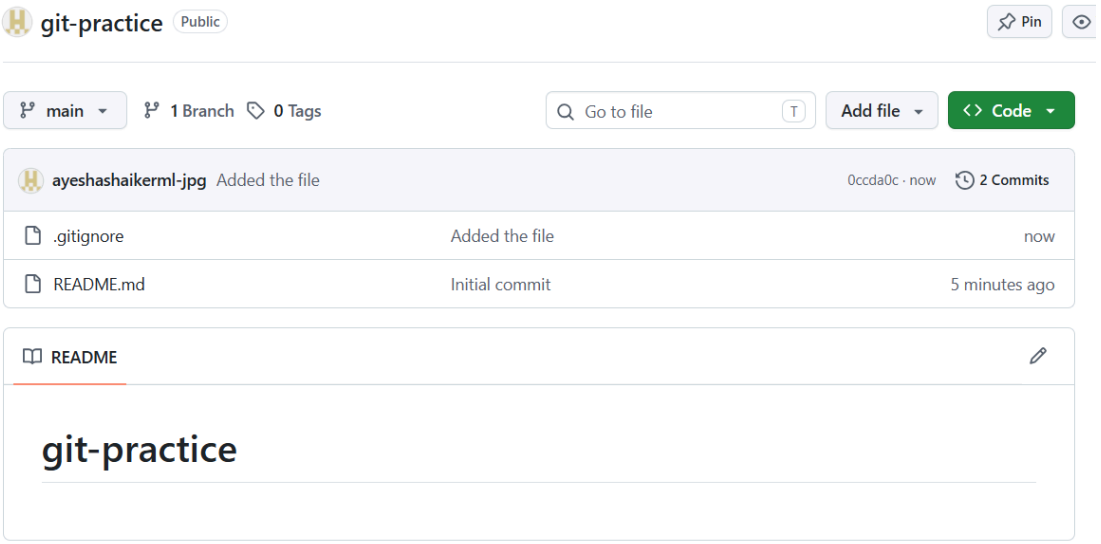
Steps

1. I install Git using the package manager.
2. I use git --version to verify that Git is installed.
3. If Git is installed successfully, it displays the installed Git version.

2. Create a GitHub repository with README.md and .gitignore

Commands

There are no Linux commands required for creating the GitHub repository. I create it from the GitHub website.



Steps

1. I log in to GitHub.
2. I select **New repository**.
3. I give the repository a name, for example:
   git-practice
4. I select:
   Add a README file
5. I select or create a .gitignore file.
6. I create the repository.

3. Clone the GitHub repository to local

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~ (master)
$ git clone git@github.com:ayeshashaikerml-jpg/git-practice.git
Cloning into 'git-practice'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~ (master)
$ cd git-practice

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ touch .gitignore

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ ls -la
total 21
drwxr-xr-x 1 Ayesha 197121  0 Oct 23 14:34 ./
drwxr-xr-x 1 Ayesha 197121  0 Oct 23 14:33 ../
drwxr-xr-x 1 Ayesha 197121  0 Oct 23 14:33 .git/
-rw-r--r-- 1 Ayesha 197121  0 Oct 23 14:34 .gitignore
-rw-r--r-- 1 Ayesha 197121 14 Oct 23 14:33 README.md

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git add .

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore


Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git commit -m "Added the file"
[main 0ccda0c] Added the file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .gitignore

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 285 bytes | 285.00 KiB/s, done.
```

Steps

1. I copy the repository URL from GitHub.
2. I use git clone to download the repository to my Linux machine.
3. I enter the repository directory using cd.
4. I use ls -la to see the files, including the hidden .git directory.
5. I use git status to check the current repository status.

4. Create two files in the local repository

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ touch file1 file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ ls
README.md  file1  file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file1
        file2

nothing added to commit but untracked files present (use "git add" to track)

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$
```

Steps

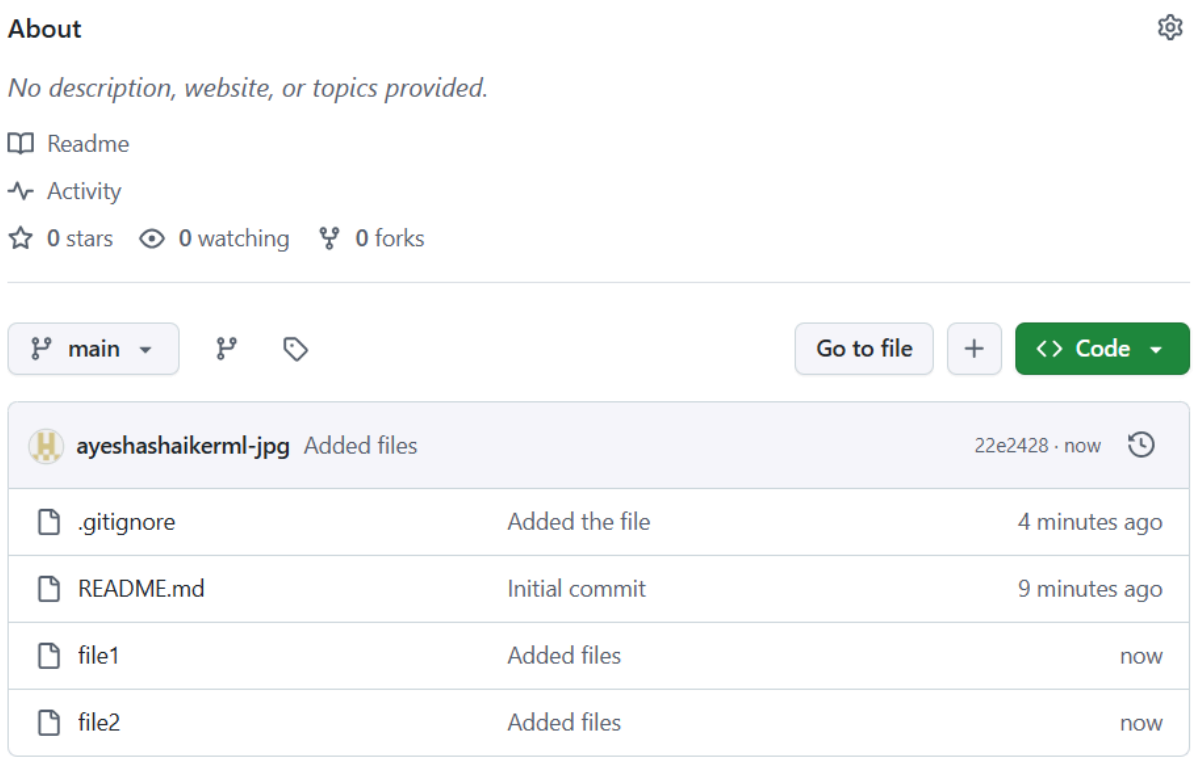
1. I make sure I am inside my local Git repository.
2. I create two files using touch.
3. I add sample content to both files.
4. I run git status.
5. Git shows these files as **untracked** because I have not added them to the staging area yet.

5. Commit two files and push them to the central repository

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git add file1 file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git commit -m "Added files"
[main 22e2428] Added files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1
 create mode 100644 file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 287 bytes | 287.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:ayeshashaikerml-jpg/git-practice.git
   0ccda0c..22e2428  main -> main
```



Steps

1. I check the status using git status.
2. I move both files to the staging area using:
   git add file1.txt file2.txt
3. I check the status again to confirm that they are staged.

4. I create a commit:
   git commit -m "Added two files"
5. The commit is now stored in my **local repository**.
6. I push the commit to GitHub:
   git push -u origin main
7. Now the files and commit are available in the GitHub repository.

6. Create a branch locally, create a file and push it to GitHub

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git branch release

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git branch --list
* main
  release

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git checkout release
Switched to branch 'release'

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ touch ayesha

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ ls
README.md  ayesha  file1  file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git add .

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git status
On branch release
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   ayesha


Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git push
fatal: The current branch release has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin release

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.


Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git commit -m "file added"
[release 2354830] file added
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 ayesha

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git push -u origin release
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 239 bytes | 239.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
```

Steps

1. I created a local branch named **release**.
2. I switched from the **main** branch to the **release** branch.
3. I created a sample file named **ayesha** inside the release branch.
4. I checked the files and confirmed that ayesha was created.

5. I checked the Git status and confirmed that ayesha was an untracked file.
6. I added the ayesha file to the **staging area**.
7. I committed the ayesha file to my **local repository** with a suitable commit message.
8. I pushed my local **release branch** to the GitHub remote repository.
9. I verified in GitHub that the **release branch** and the **ayesha file** are available.

7. Create a branch in GitHub and get that branch locally

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git fetch origin
From github.com:ayeshashaikerml-jpg/git-practice
 * [new branch]      develop    -> origin/develop

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git branch --list
  main
* release

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git switch -c develop --track origin/develop
Switched to a new branch 'develop'
branch 'develop' set up to track 'origin/develop'.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (develop)
$ git branch --list
* develop
  main
  release

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (develop)
$
```

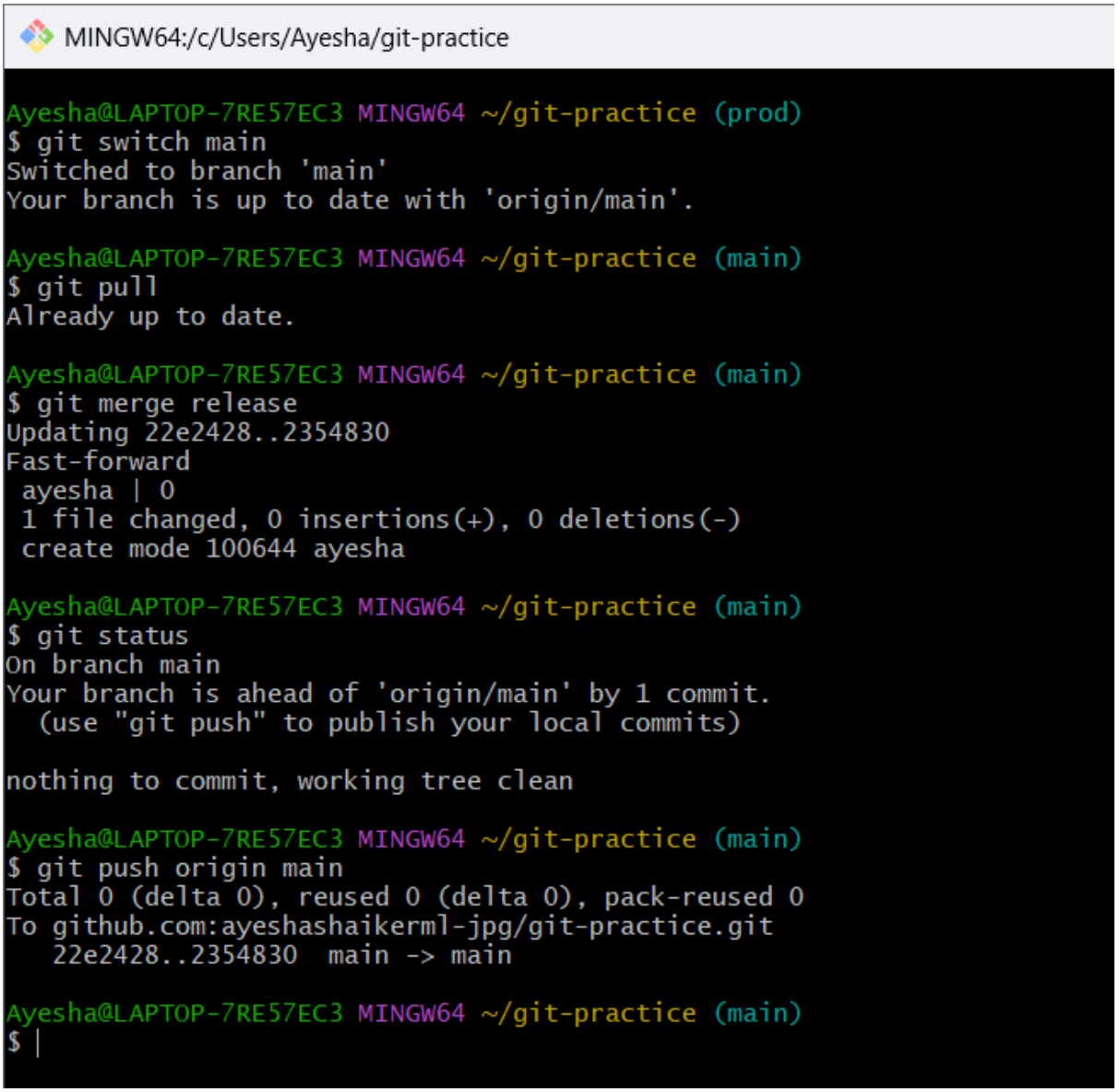
Step 1: Update local reference list

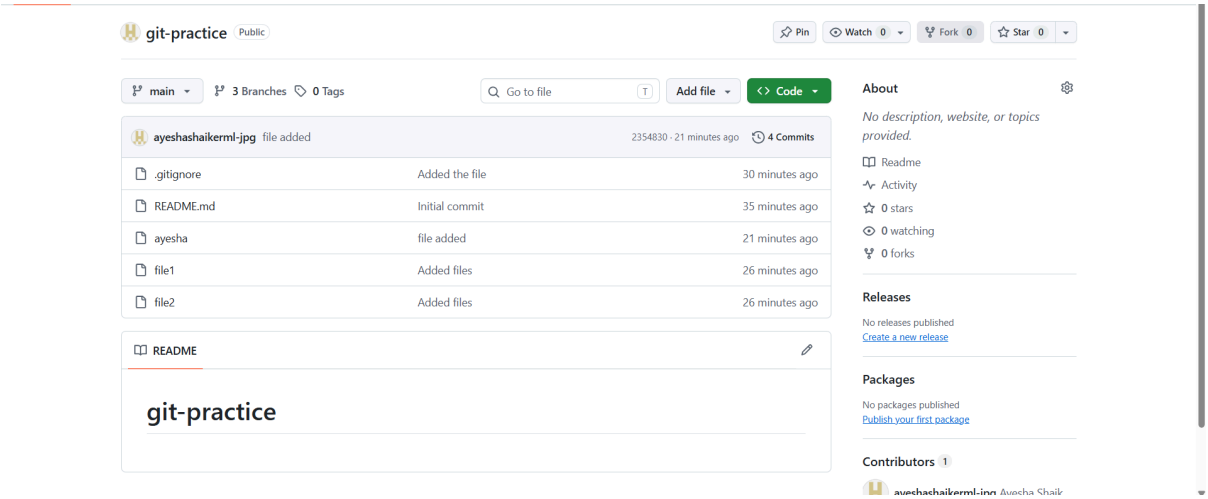
Run a fetch to let your local Git repository know that the new prod branch exists on the remote server

Step 2: Switch to the new branch

Now that your local Git sees the remote branch, you can switch to it. This command automatically creates a local branch named prod that tracks the remote origin/prod branch:

8. Merge a branch with main locally





I first check my available branches and confirm that I have main and release.

I switch from the release branch to the main branch because I want to merge the changes **into main**.

I update my local main branch with the latest changes from the GitHub repository.

I merge the release branch into my current main branch.

Git combines the changes from release with main.

If there are no conflicts, the merge completes successfully.

I check the repository status to make sure the merge was successful.

I check the commit history to verify that the changes from release are now part of main.

Finally, I push the updated main branch to GitHub so that the merged changes are available in the remote repository.

9. Merge a branch with main using GitHub Pull Request

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git checkout prod
Switched to branch 'prod'
Your branch is up to date with 'origin/prod'.

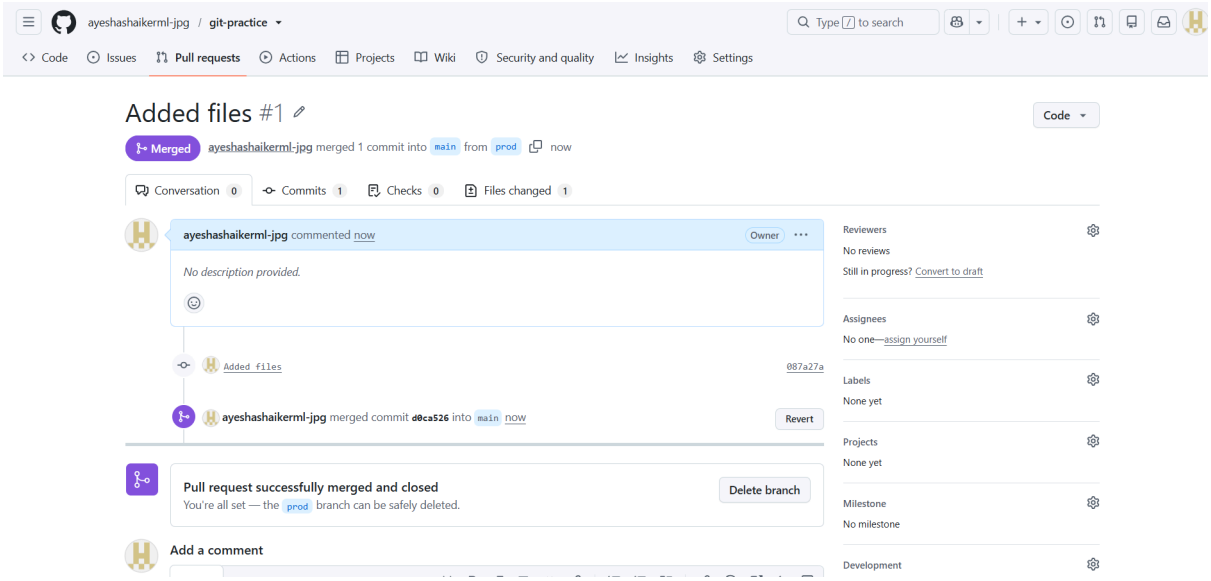
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ touch feature1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git add .

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git commit -m "Added files"
[prod 087a27a] Added files
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git push -u origin prod
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 240 bytes | 240.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ayeshashaikerml-jpg/git-practice.git
   22e2428..087a27a  prod -> prod
branch 'prod' set up to track 'origin/prod'.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$
```



```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 940 bytes | 117.00 KiB/s, done.
From github.com:ayeshashaikerml-jpg/git-practice
 * [new branch]      feature    -> origin/feature
   2354830..d0ca526  main       -> origin/main

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
  (use "git pull" to update your local branch)

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ ls
README.md  ayesha  file1  file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git pull
Updating 2354830..d0ca526
Fast-forward
 feature1 | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ ls
README.md  ayesha  feature1  file1  file2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$
```

1. I switch to the branch where I want the file.
2. I create the file.
3. I add content.
4. I stage the file.
5. I commit the file.
6. I push the branch to GitHub.
7. The file now appears inside the prod branch on GitHub.
8. I create the Pull Request.
9. I review the changes.
10. After approval, I select **Merge pull request**.
11. GitHub merges prod into main.

10. Create a file locally and send it to a branch in GitHub

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:ayeshashaikerml-jpg/git-practice.git
   22e2428..2354830  main -> main

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git checkout prod
Switched to branch 'prod'
Your branch is up to date with 'origin/prod'.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ touch feature1

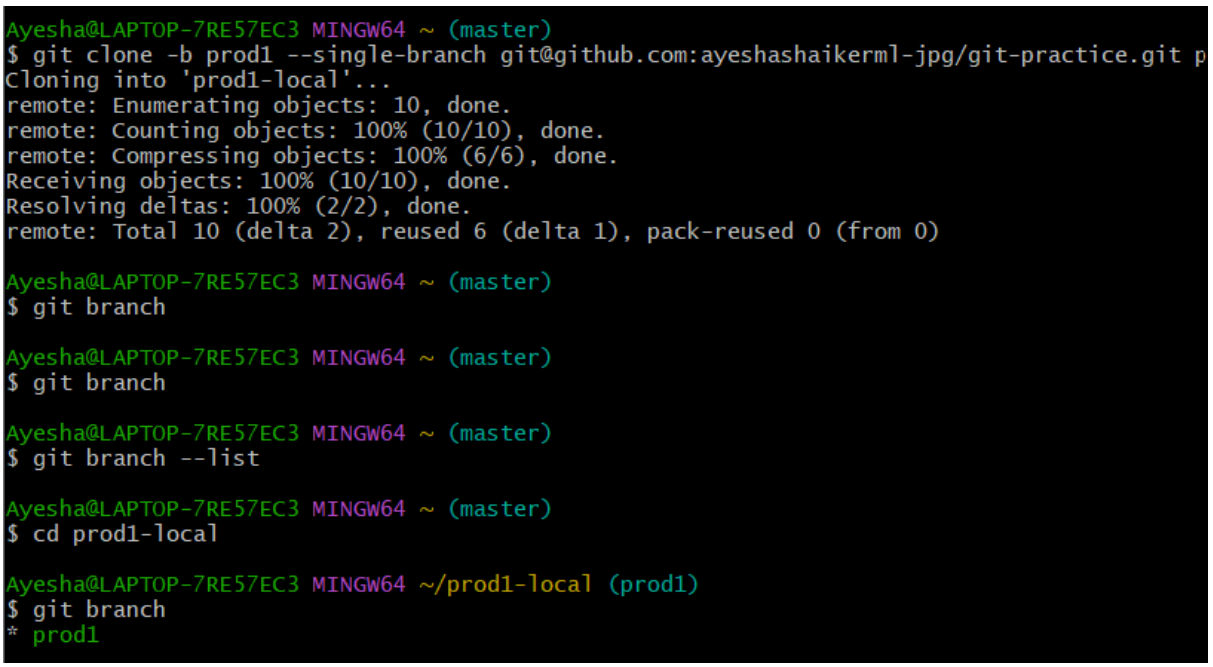
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git add .

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git commit -m "Added files"
[prod 087a27a] Added files
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (prod)
$ git push -u origin prod
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 240 bytes | 240.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
```

1. I switch to the branch where I want the file.
2. I create the file.
3. I stage the file.
4. I commit the file.
5. I push the branch to GitHub.
6. The file now appears inside the prod branch on GitHub.

11. Clone only one branch from GitHub



I created the prod1 branch on GitHub.

I moved outside my existing git-practice repository before performing a new clone.

I attempted to clone only the prod1 branch.

Git initially showed an error because the git-practice destination directory already existed.

I avoided deleting my existing repository because it contained my previous practical work.

I chose a different local destination directory named prod1-local.

I successfully cloned the repository using the prod1 branch and the single-branch option.

Git created the new prod1-local directory.

I entered the prod1-local directory.

I checked the current branch.

I verified that the local repository was on the prod1 branch.

12. Create a password file and make Git ignore it

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touche

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-practice/


Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git commit -m "Add gitignore for password file"
[main 10296a4] Add gitignore for password file
 1 file changed, 1 insertion(+)

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git oush
git: 'oush' is not a git command. See 'git --help'.

The most similar command is
        push

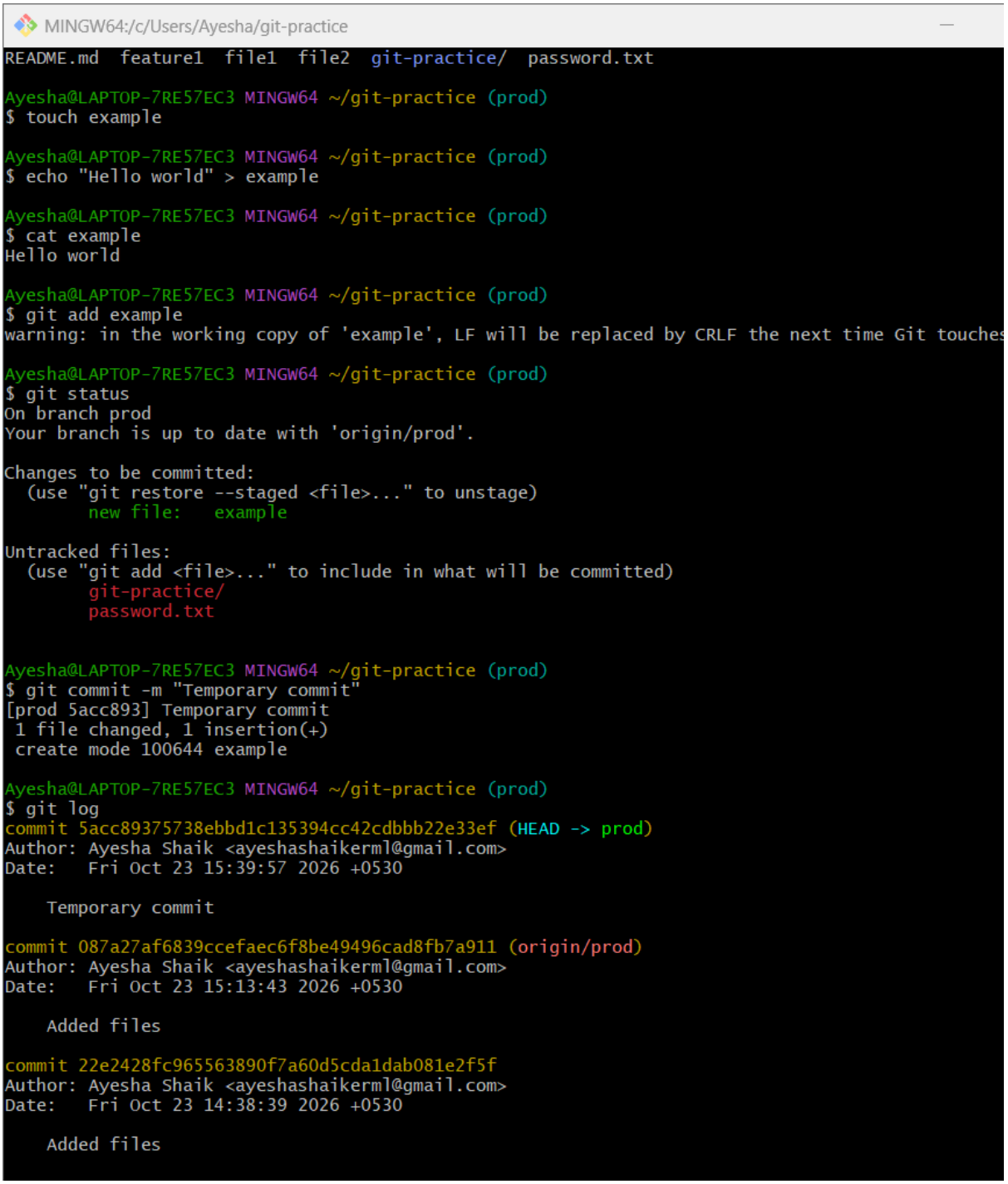
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 308 bytes | 308.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ayeshashaikerml-jpg/git-practice.git
   d0ca526..10296a4  main -> main

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (main)
$
```

Steps

1. I create a file called passwords.txt.
2. For practice, I add dummy credentials.
3. I add passwords.txt to .gitignore.
4. I check git status.
5. Git no longer shows the untracked password file.

13. Commit and reset the commit without keeping its changes

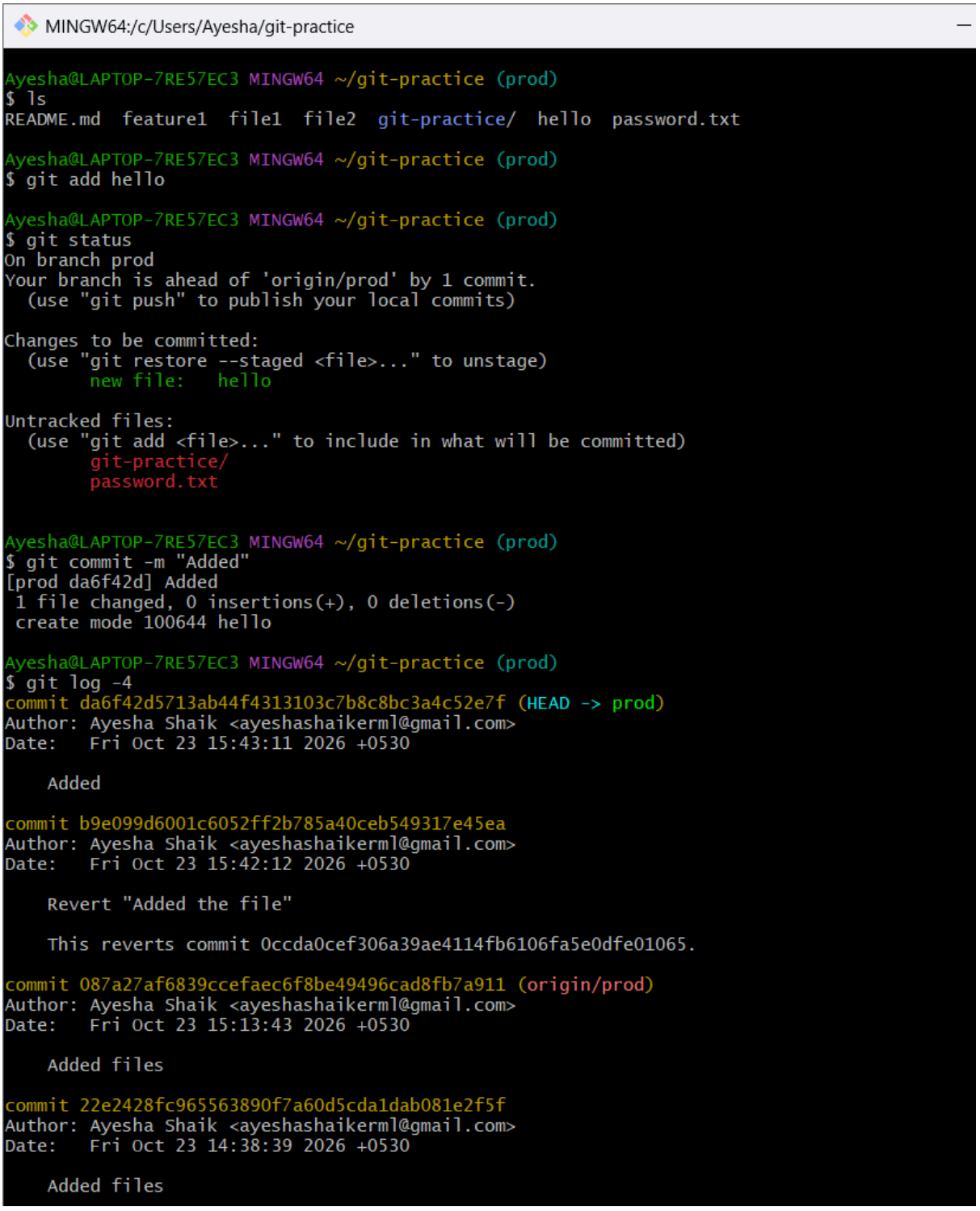


Steps

1. I create a temporary file.
2. I stage it.
3. I commit it.
4. I check the commit history.
5. I use:
   git reset --hard HEAD~1

6. Git moves my current branch back by one commit.
7. The commit and its changes are removed from my working directory.

14. Revert a committed commit to the older version



Steps

1. I check my commit history.
2. I identify the commit that I want to undo.
3. I run:
   git revert COMMIT_ID
4. Git creates a new commit that reverses the selected commit.
5. I check the history.
6. I push the new revert commit to GitHub.

15. Stash changes and work on another file

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git stash -u
Ignoring path git-practice/
warning: in the working copy of 'password.txt', LF will be replaced by CRLF the next
t
Saved working directory and index state WIP on release: 2354830 file added

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ ls
README.md  ayesha  file1  file2  git-practice/

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ echo "Hey" file1
Hey file1

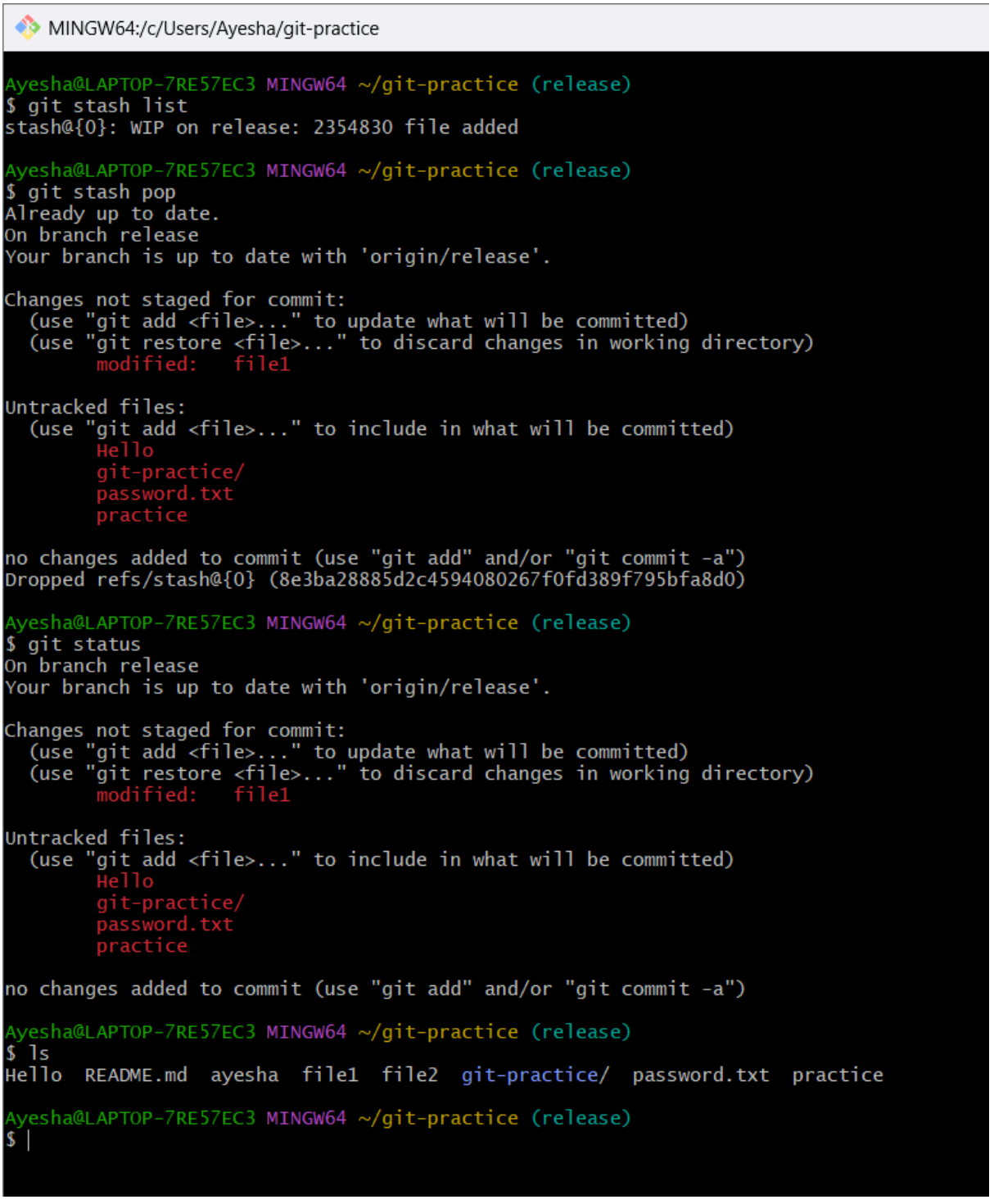
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ echo "Hey" > file1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ cat file1
Hey
```

Steps

1. I start modifying Hello and untracked files.
2. I am not ready to commit those changes.
3. I need to work on another file.
4. I use git stash when I have unfinished work that I don't want to commit yet but need a clean working directory for another task.

16. Restore the stashed changes

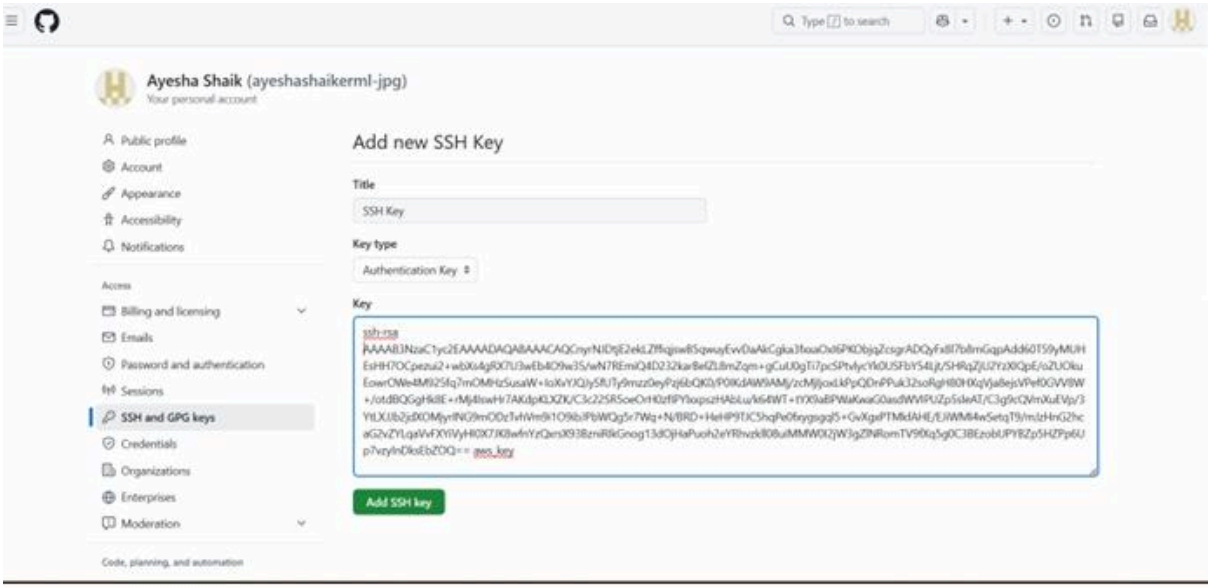


Steps

1. I check my available stashes:
   git stash list
2. I restore my latest stash:
   git stash pop

3. Git applies my previous changes back to the working directory.
4. I can now continue working on the file.

17. Generate SSH key and configure GitHub




```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/hiring-app (main)
$ git push -u origin main
The authenticity of host 'github.com (20.207.73.82)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/hiring-app (main)
$ ls -la ~/.ssh
total 44
drwxr-xr-x 1 Ayesha 197121    0 Oct 22 09:41 ./
drwxr-xr-x 1 Ayesha 197121    0 Oct 22 15:01 ../
-rw-r--r-- 1 Ayesha 197121 3369 Oct 12 15:10 id_rsa
-rw-r--r-- 1 Ayesha 197121  733 Oct 12 15:10 id_rsa.pub
-rw-r--r-- 1 Ayesha 197121 4596 Oct 22 15:02 known_hosts
-rw-r--r-- 1 Ayesha 197121 4332 Oct 22 09:41 known_hosts.old

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/hiring-app (main)
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQCnyrNJDtjE2ekLZffiqjswB5qwuyEvvDaAkCgka3fxxaOxI6PKObjqZcsgrADQyF
817b8mGqpAdd60T59yMUHEsHH7OCpezui2+wbXs4gRX7U3wEb4O9w35/wN7REmiQ4D232karBelZL8mZqm+gCuUOgTi7pcSPtvlycY
OUSFbY54Ljt/SHRqZjU2YzXIQpE/oZUOkuEowrOWe4M925fq7mOMHzSusaW+IoXvYJQJySfUTy9mzzOeyPzj6bQKO/P0IKdAW9AMj/
cMjIjoxLkPpQDnPPuk32soRgH8OHXqVja8ejsVPef0GVV8W+/otdBQGgHk8E+rMj4IswHr7AKdpKLXZK/C3c22SR5oeOrH0zfIPYlx
pszHAbLu/k64WT+tYX9aBPWaKwaGOasdWVIPUZp5sleAT/C3g9cQVmXuEVp/3YtLXJJb2jdXOMjyrING9mODzTvhVm9i1O9ibJPbWQ
5r7Wq+N/BRD+HeHP9TJC5hqPeOfxygsgql5+GvXgxPTMkfAHE/EJiWMI4wSetqT9/mJzHnG2hcaG2vZYLqaVvFXYiVyHIOX7JK8wfn
zQxrsX93BzniRIkGnog13dOjHaPuoh2eYRhvzkll08uiMMWOI2jW3gZlNRomTV9fXq5gOC3BEzobUPYBZp5HZPp6Up7vzyInDksEbZ
Q== aws_key

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/hiring-app (main)
$
```

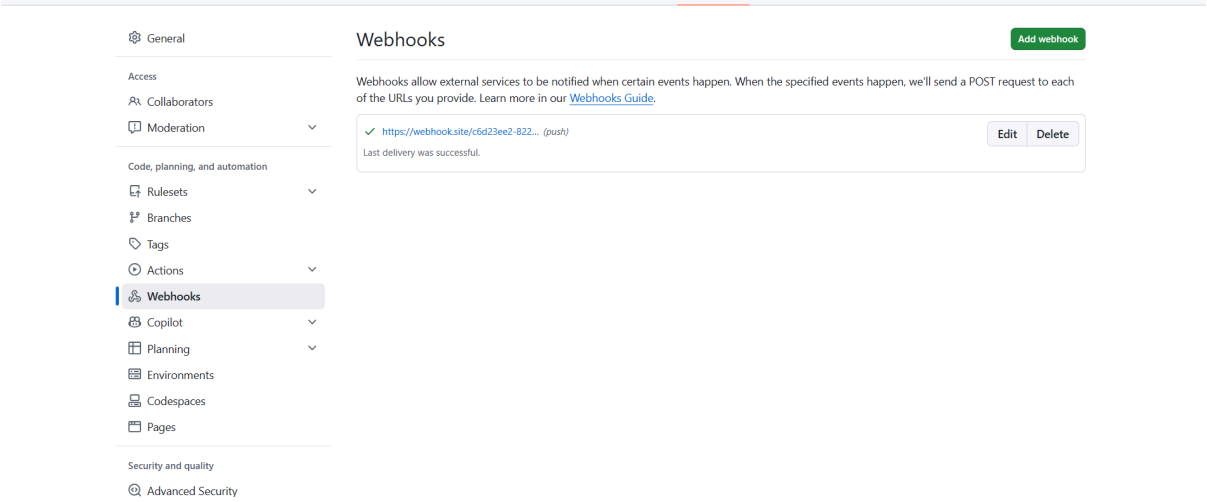
Steps

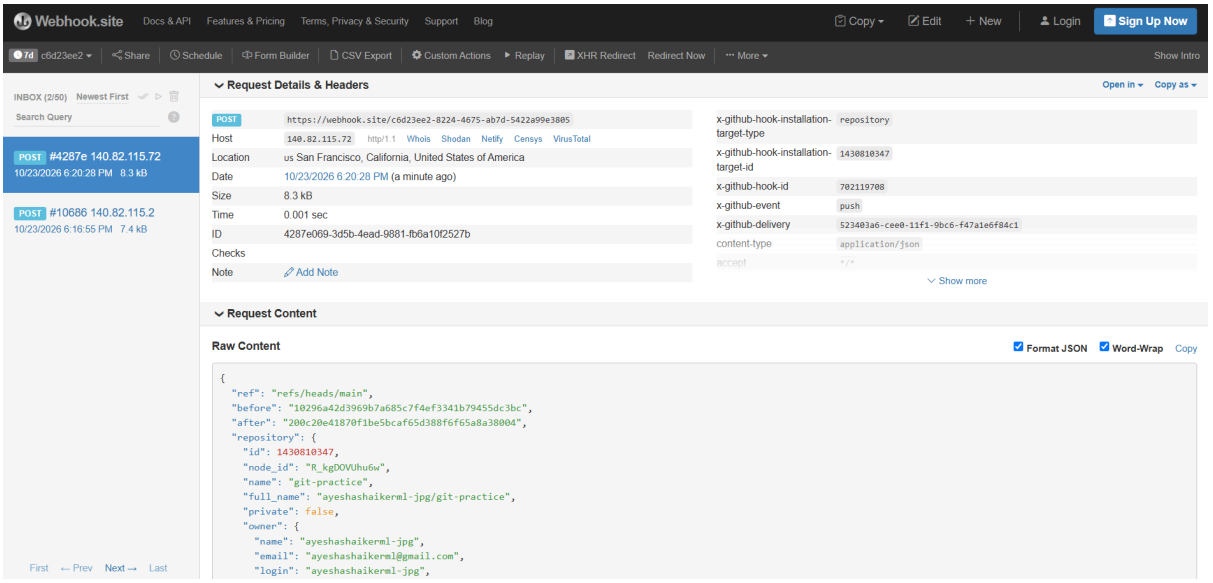
1. I generate an SSH key pair.
2. I add my public key to the SSH agent.
3. I display my public key.
4. I copy the contents of public key
5. Added in my github account ssh and Ghc keys and add public key

18. Configure GitHub Webhook

Commands

There is no normal Git command required to create the GitHub webhook.





Steps to test the GitHub Webhook

1. **Open your local Git repository** on your computer.
2. **Make a small change** in the repository.
   For example, create a new file or modify an existing file.
3. **Save the change** in your local repository.
4. **Stage the changed file** so Git knows which change should be included.
5. **Create a commit** with a meaningful commit message.
6. **Push the commit to GitHub.**
   This is the important step because the webhook is configured to respond to the GitHub push event.
7. After GitHub receives the push, **GitHub detects the push event**.
8. GitHub automatically creates an **HTTP POST request** containing information about the event.
9. GitHub sends this request to the **Payload URL** that you configured in the webhook.
10. In our practice setup, the Payload URL belongs to **Webhook.site**, so Webhook.site receives the request.
11. **Open Webhook.site** and check the incoming request.
12. Open the received request to view the **JSON payload** sent by GitHub.

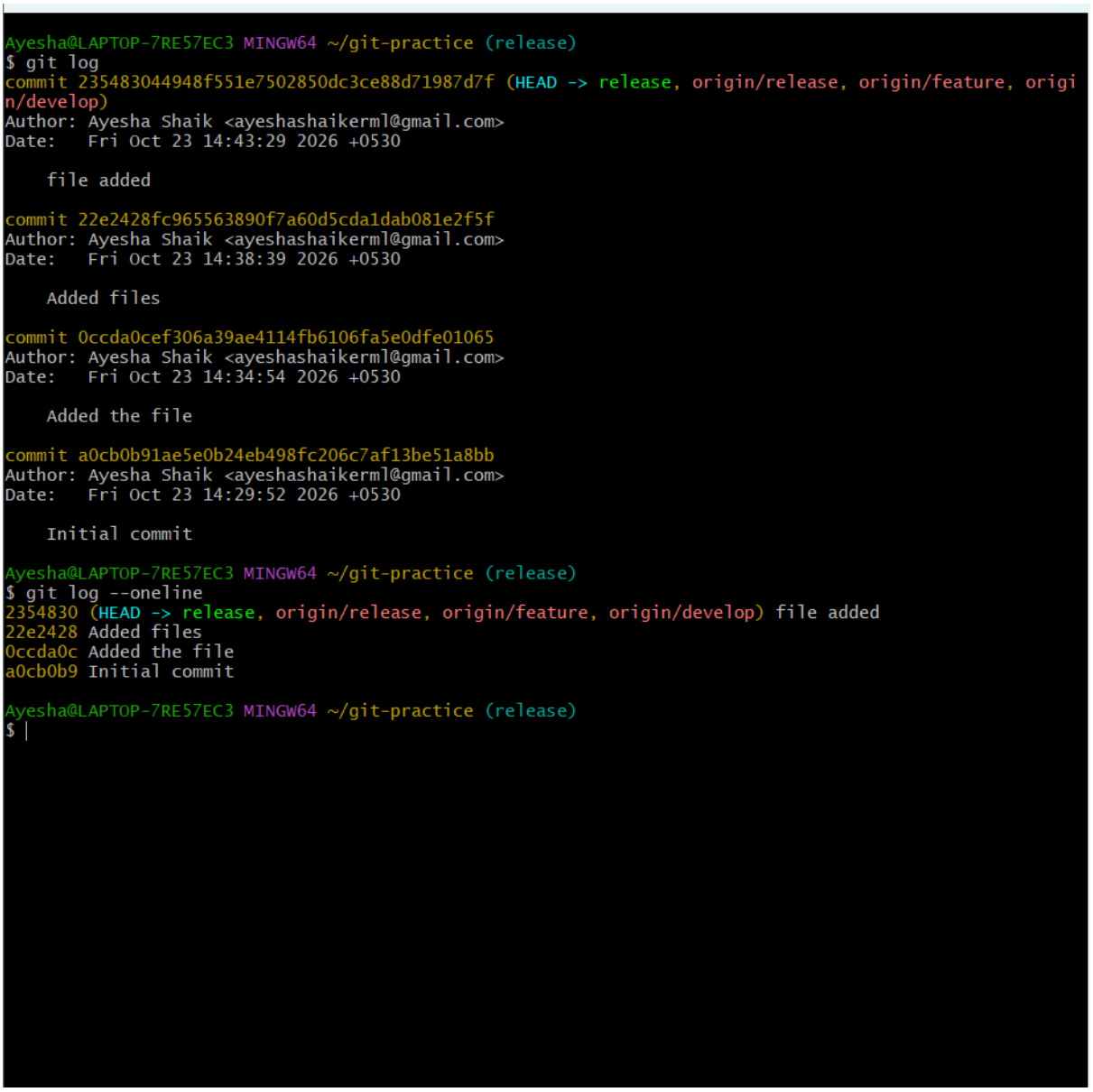
19. Understand the .git directory

The .git directory is the heart of the Git repository. It stores all the information Git needs to track the project, maintain commit history, manage branches, and maintain the staging area.

The actual project files are outside the .git directory.

When I execute git init inside a project directory, Git creates a hidden directory named **.git**. The .git directory contains all the internal information required for Git to manage my repository, such as commit history, branches, configuration, and tracking information. I normally do not modify the contents of the .git directory manually. If I delete the .git directory, the project will no longer be treated as a Git repository, although my actual project files will remain.

20. Check Git logs



I used Git log to check the commit history of my local repository. It displays the commit ID, author, date, and commit message. I can also use git log --oneline to view the commit history in a compact format.

-

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git commit --amend -m "correct commit message"
[release 90d8306] correct commit message
 Date: Fri Oct 23 14:43:29 2026 +0530
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 ayesha

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git log
commit 90d8306b7ad15b321096921b929a5c0e996416c7 (HEAD -> release)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:43:29 2026 +0530

    correct commit message

commit 22e2428fc965563890f7a60d5cda1dab081e2f5f
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:38:39 2026 +0530

    Added files

commit 0ccda0cef306a39ae4114fb6106fa5e0dfe01065
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:34:54 2026 +0530

    Added the file

commit a0cb0b91ae5e0b24eb498fc206c7af13be51a8bb
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:29:52 2026 +0530

    Initial commit

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$
```

Normal Steps

Step 1: Check the latest commit

I first check my Git commit history and identify the latest commit.

Step 2: Identify the commit message

I look at the message of the latest commit and decide what I want to change.

For example:

Old message:

Add file

Step 3: Amend the latest commit

I use Git's **amend** operation to modify the latest commit.

When I amend the commit, I can provide the new commit message.

22. Merge multiple commits into one commit

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git log
commit c60081951d59c3c58d1e1a5e4c6702b036ed0d6d (HEAD -> release)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 16:22:39 2026 +0530

    Added all

commit 90d8306b7ad15b321096921b929a5c0e996416c7
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:43:29 2026 +0530

    correct commit message

commit 22e2428fc965563890f7a60d5cda1dab081e2f5f
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:38:39 2026 +0530

    Added files

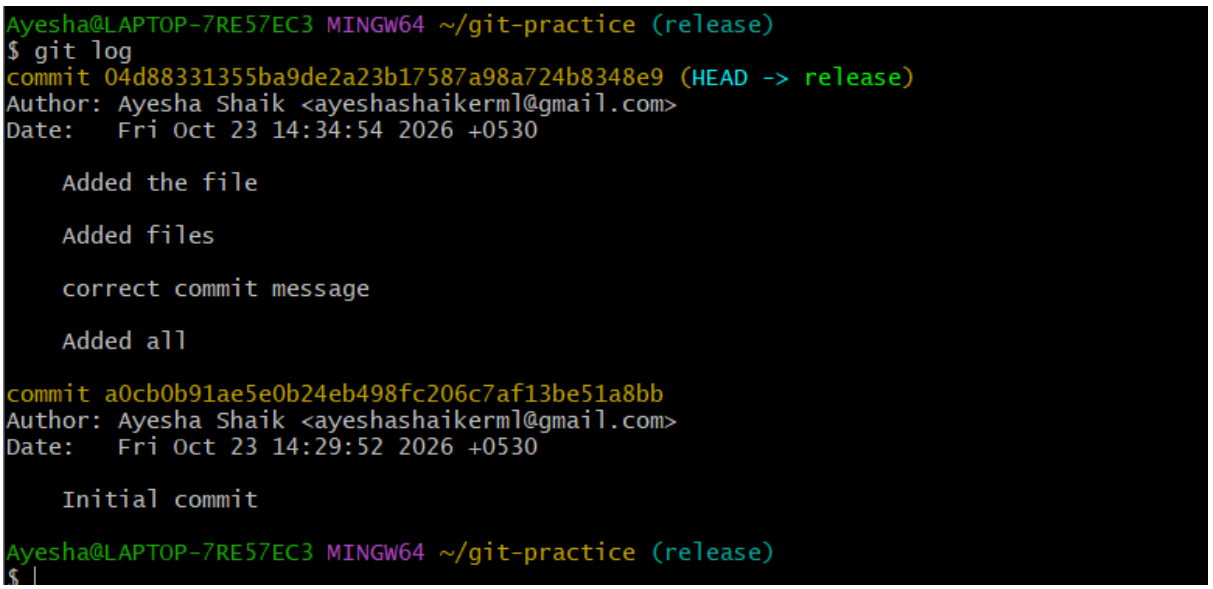
commit 0ccda0cef306a39ae4114fb6106fa5e0dfe01065
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:34:54 2026 +0530

    Added the file

commit a0cb0b91ae5e0b24eb498fc206c7af13be51a8bb
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date:   Fri Oct 23 14:29:52 2026 +0530

    Initial commit

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git-practice (release)
$ git rebase -i HEAD~4
warning: unable to rmdir 'git-practice': Directory not empty
[detached HEAD 04d8833] Added the file
 Date: Fri Oct 23 14:34:54 2026 +0530
 8 files changed, 3 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Hello
 create mode 100644 ayesha
 create mode 100644 file1
 create mode 100644 file2
 create mode 160000 git-practice
 create mode 100644 password.txt
 create mode 100644 practice
Successfully rebased and updated refs/heads/release.
```



Create multiple commits for the same task.

Check the commit history and identify the commits that I want to combine.

Select the number of recent commits that I want to merge into one.

Start an interactive rebase for those commits.

Git displays the selected commits in order.

Keep the first commit as the main commit.

Mark the remaining commits as "squash".

Git combines the remaining commits with the first commit.

Git asks me to choose the final commit message.

I remove the unnecessary old messages and provide one meaningful message.

Git completes the rebase and creates one combined commit.

Check the Git history again and verify that the multiple commits have become one commit.